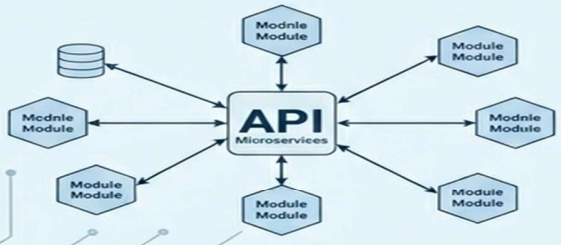
Clinic Management System



Prepared by: Team [36]
Faculty of Computer Science
2026

Software Requirements Specification (SRS)

the Clinic Management System

Software Requirements Specification (SRS)

Project: Clinic Management System

Architecture: Microservices Architecture

1. Introduction

The Clinic Management System is a microservices-based healthcare platform designed to manage patients, doctors, appointments, medical records, and authentication securely and efficiently. The system provides role-based access for Admins, Doctors, Receptionists, and Patients.

1.1 Purpose

This document defines the functional and non-functional requirements for the Clinic Management System. It serves as a guide for developers, testers, and stakeholders.

1.2 Scope

The system enables patient registration, appointment booking, medical record management, authentication, and audit logging through separate microservices connected via an API Gateway.

2. Overall Description

The system follows a microservices architecture where each service is independent and communicates through REST APIs.

2.1 User Roles

- Admin: manages users, doctors, and patients.
- Doctor: manages appointments and medical records.
- Receptionist: handles patient registration and appointment booking.
- Patient: views appointments and medical history.

2.2 System Architecture

The backend is implemented using Spring Boot microservices including:
- Auth Service
- Patient Service
- Doctor Service
- Appointment Service
- Medical Record Service
- Audit Service
- API Gateway

3. Functional Requirements

The system shall support authentication, role-based authorization, appointment management, patient management, doctor management, and medical records.

3.1 Authentication Requirements

- Users shall log in using email and password.
- JWT token shall be generated after successful login.
- Unauthorized users shall be denied access.

3.2 Patient Management

- Admins and Receptionists can create patient profiles.
- Patients can view and update their own profile.

3.3 Doctor Management

- Admins can create and update doctor profiles.
- Users can view available doctors.

3.4 Appointment Management

- Patients and Receptionists can book appointments.
- Doctors can update appointment status.
- Appointments can be cancelled.

3.5 Medical Records

- Doctors can create and update medical records.
- Patients can view their own medical history.

4. Non-Functional Requirements

The system must be secure, scalable, maintainable, and responsive.

4.1 Security

- JWT authentication must be implemented.
- Passwords must be encrypted.
- Role-based authorization must be enforced.

4.2 Performance

The system should respond to API requests within acceptable response time under normal load.

4.3 Scalability

Each microservice should scale independently.

5. Database Requirements

Each microservice maintains its own database tables and entities.

6. API Requirements

RESTful APIs shall be used for communication between frontend and backend services.

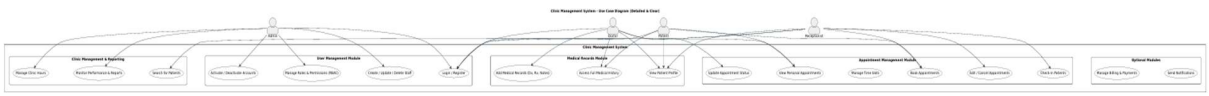
7. Future Enhancements

- Online payment integration
- Notifications and reminders
- Video consultation support

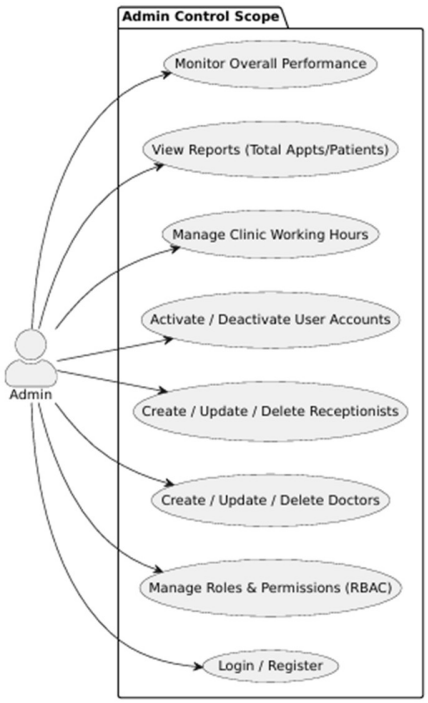
8. Diagrams Placeholder

1- Use Case Diagram

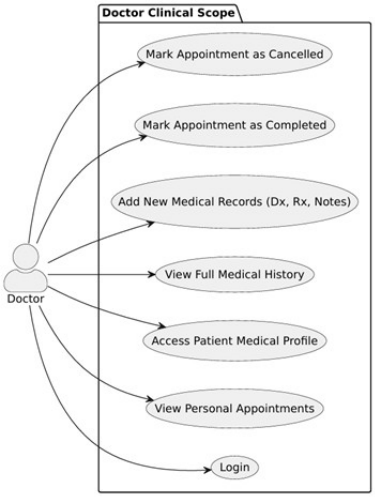
1.1- all overview Use case



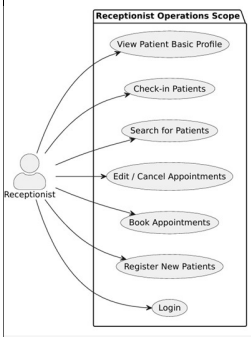
1.2- Admin Use case



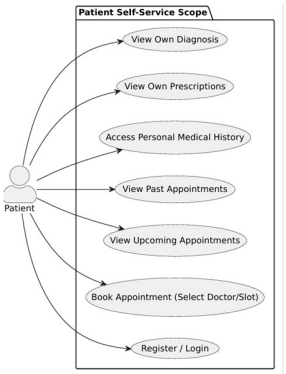
1.3- Doctor Use case



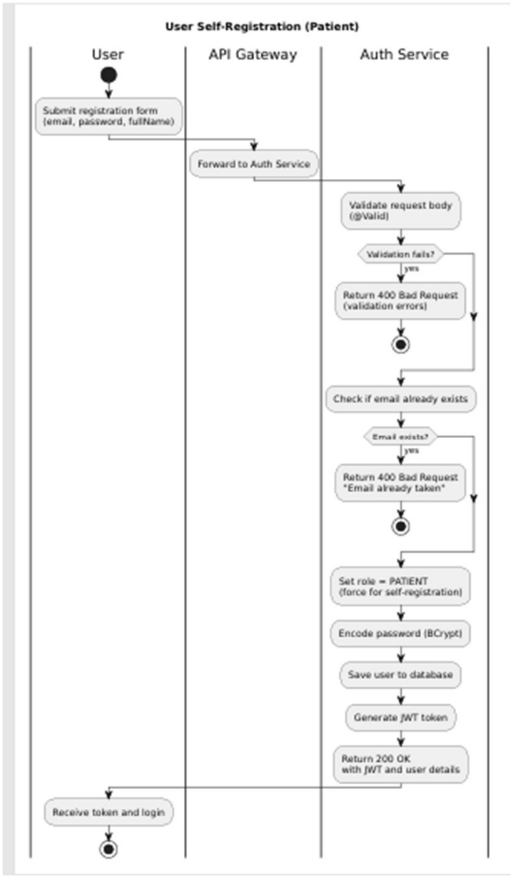
1.4- Receptionist Use case

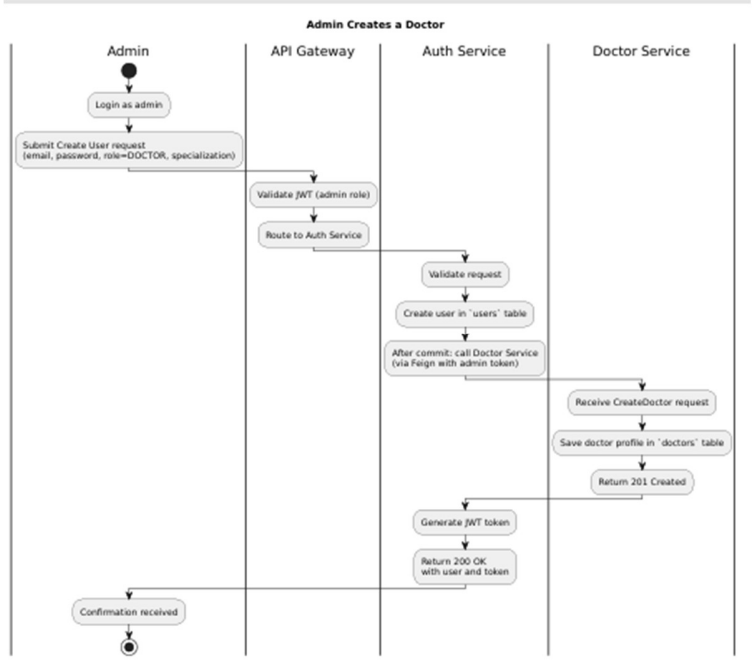


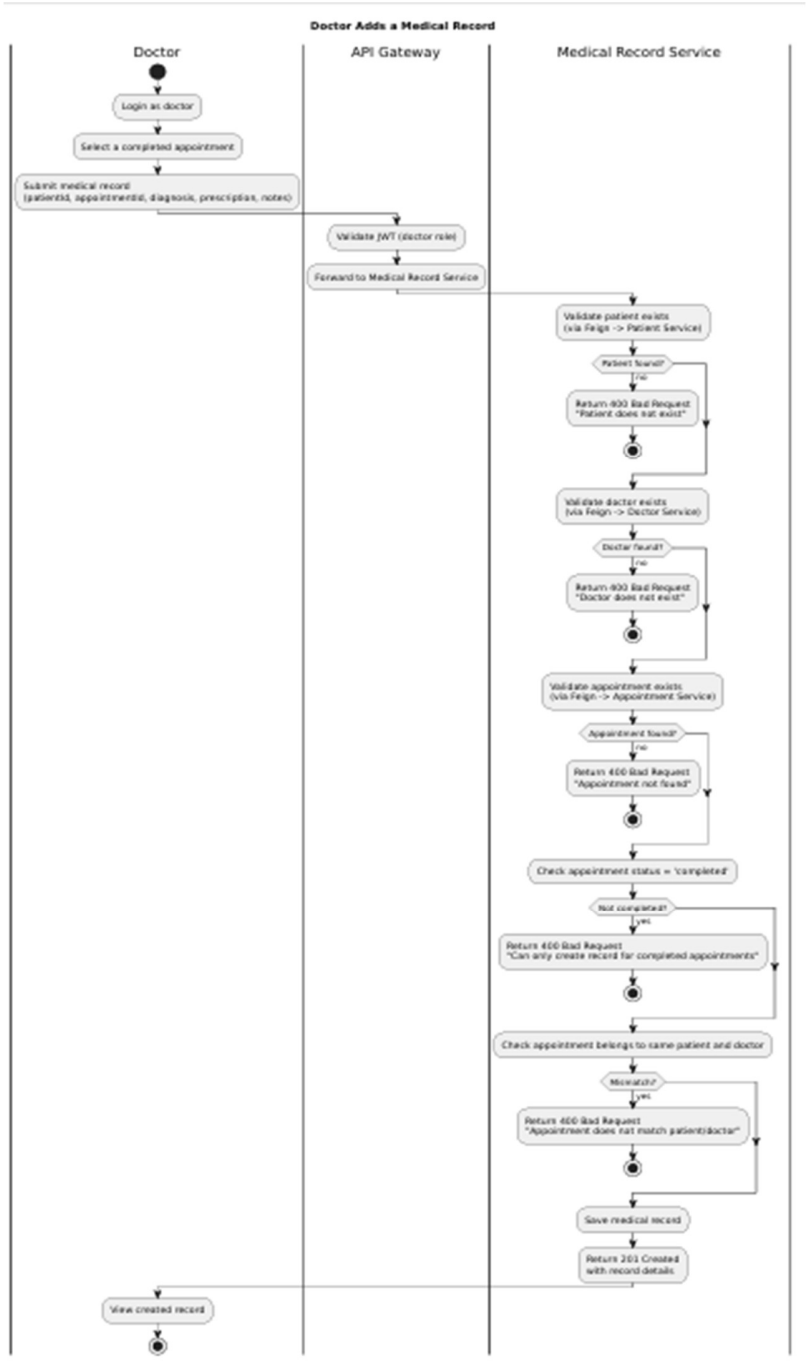
1.5- patient use case

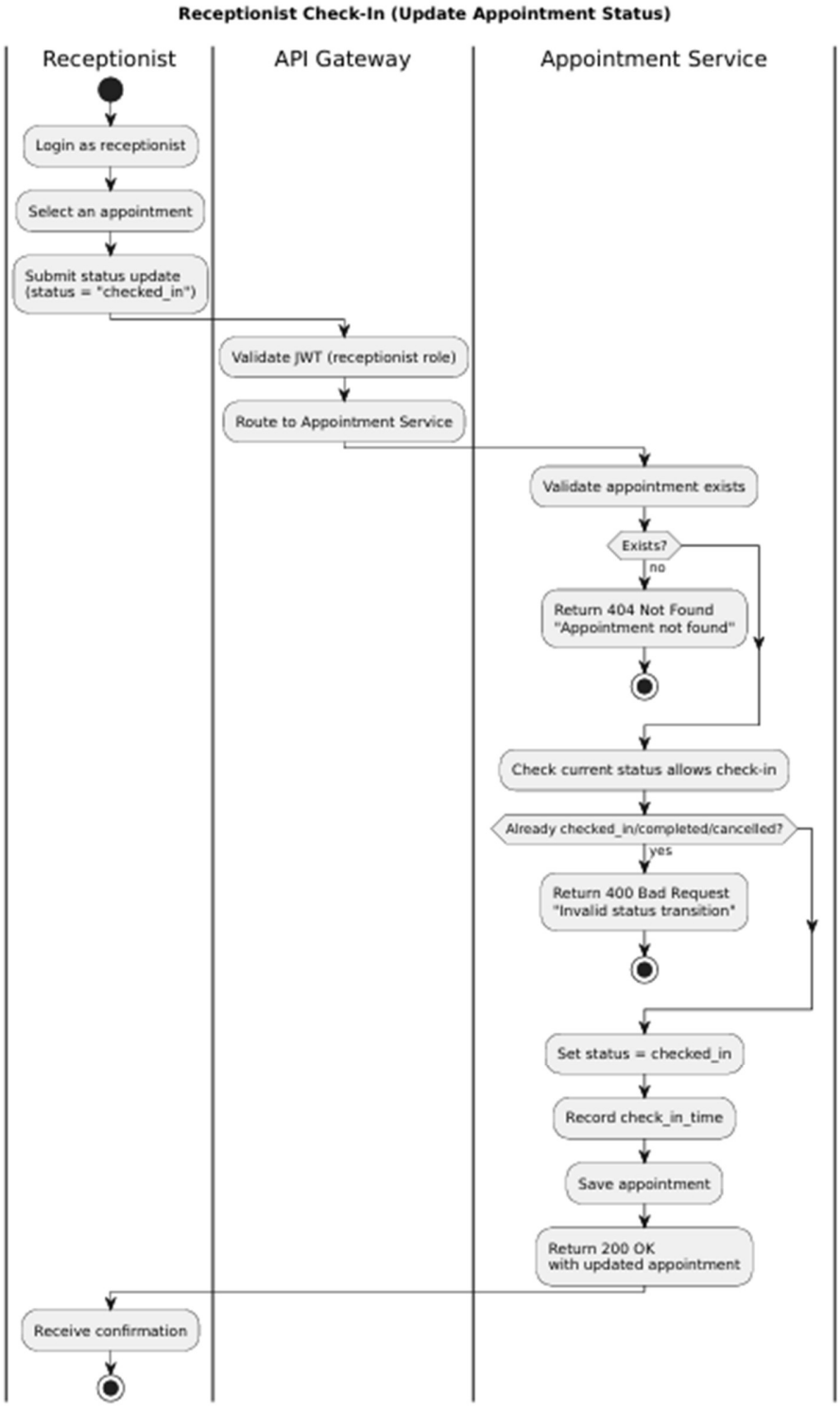


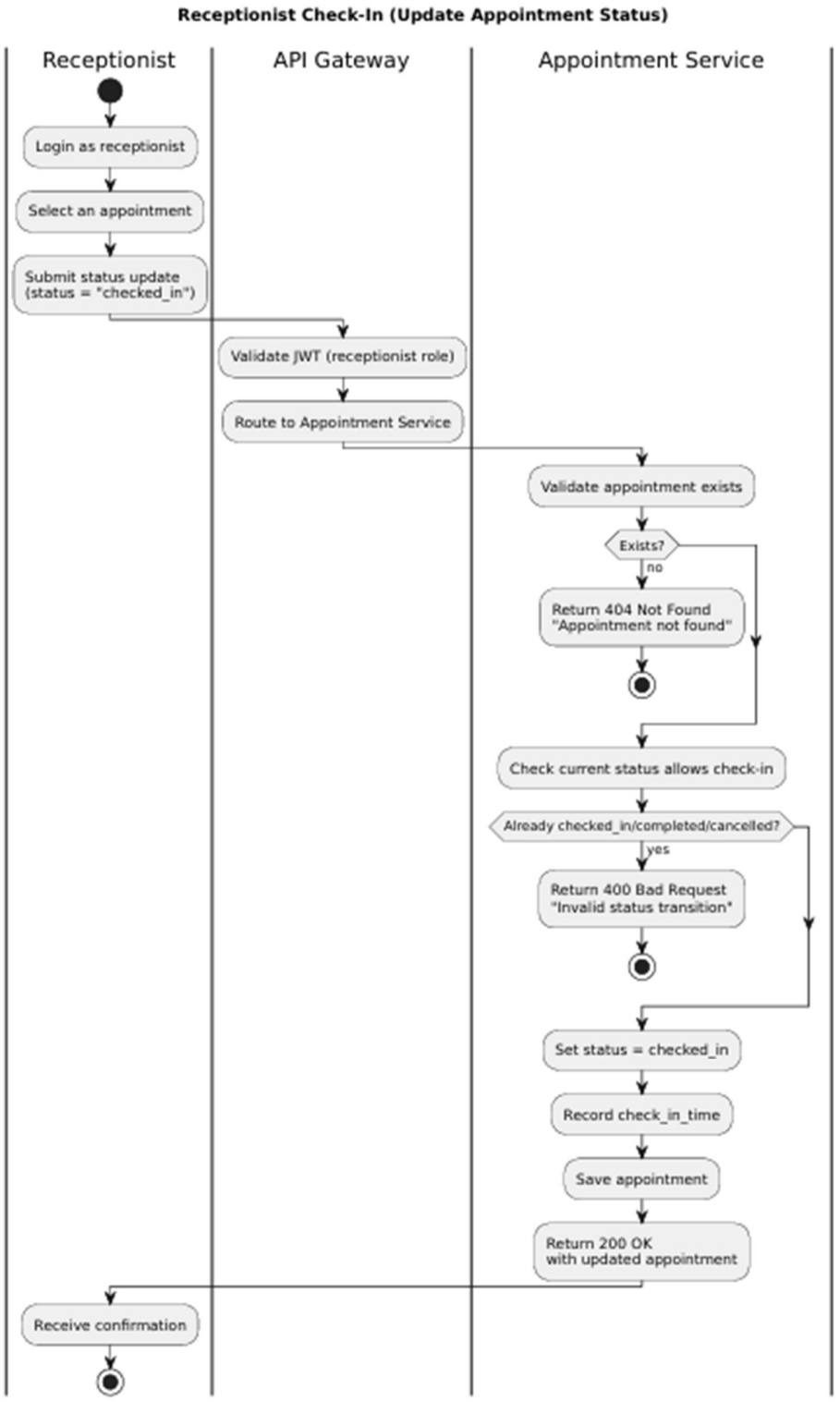
2- Activity Diagrams

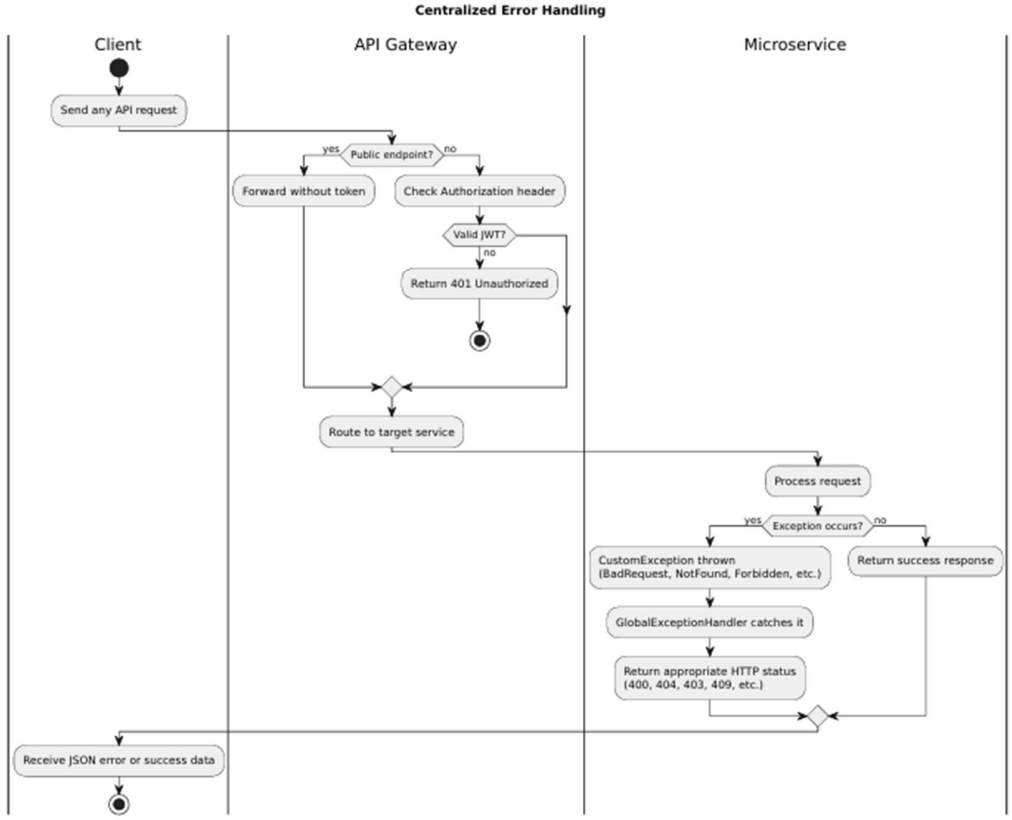

User Self-Registration (Patient)
User
API Gateway
Auth Service
Submit registration form (email, password, fullName)
Forward to Auth Service
Validate request body (@Valid)
Validation fails?
yes
Return 400 Bad Request (validation errors)
Check if email already exists
Email exists?
yes
Return 400 Bad Request "Email already taken"
Set role = PATIENT (force for self-registration)
Encode password (BCrypt)
Save user to database
Generate JWT token
Return 200 OK with JWT and user details
Receive token and login

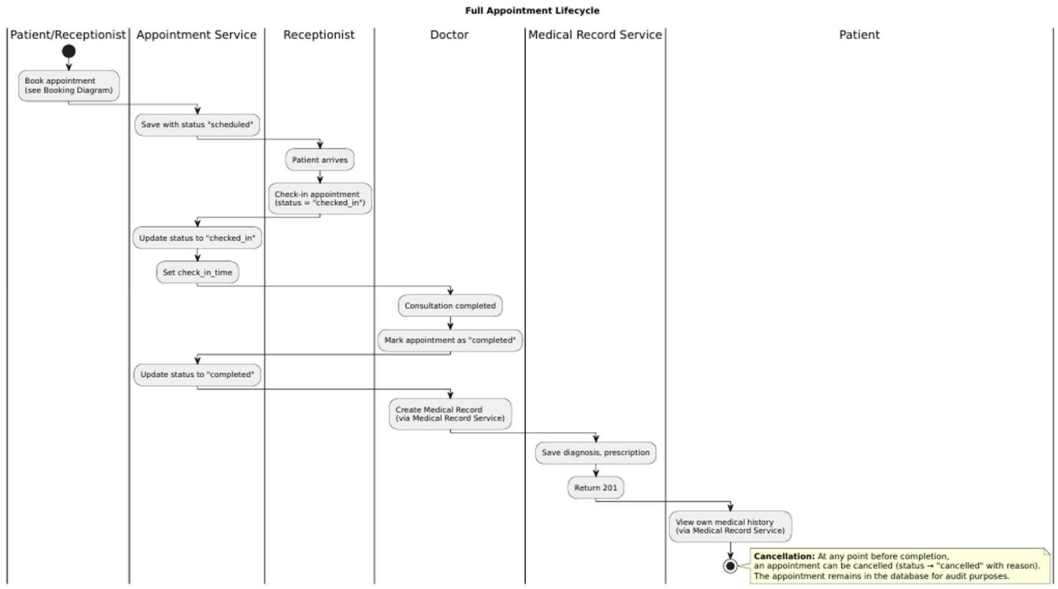

Admin Creates a Doctor
Admin
API Gateway
Auth Service
Doctor Service
Login as admin
Submit Create User request (email, password, role=DOCTOR, specialization)
Validate JWT (admin role)
Route to Auth Service
Validate request
Create user in `users` table
After commit: call Doctor Service (via Feign with admin token)
Receive CreateDoctor request
Save doctor profile in `doctors` table
Return 201 Created
Generate JWT token
Return 200 OK with user and token
Confirmation received


Doctor Adds a Medical Record
Doctor
API Gateway
Medical Record Service
Login as doctor
Select a completed appointment
Submit medical record
(patientId, appointmentId, diagnosis, prescription, notes)
Validate JWT (doctor role)
Forward to Medical Record Service
Validate patient exists
(via Feign -> Patient Service)
Patient found?
no
Return 400 Bad Request
"Patient does not exist"
Validate doctor exists
(via Feign -> Doctor Service)
Doctor found?
no
Return 400 Bad Request
"Doctor does not exist"
Validate appointment exists
(via Feign -> Appointment Service)
Appointment found?
no
Return 400 Bad Request
"Appointment not found"
Check appointment status = 'completed'
Not completed?
yes
Return 400 Bad Request
"Can only create record for completed appointments"
Check appointment belongs to same patient and doctor
Mismatch?
yes
Return 400 Bad Request
"Appointment does not match patient/doctor"
Save medical record
Return 201 Created
with record details
View created record


Receptionist Check-In (Update Appointment Status)
Receptionist
API Gateway
Appointment Service
Login as receptionist
Select an appointment
Submit status update
(status = "checked_in")
Validate JWT (receptionist role)
Route to Appointment Service
Validate appointment exists
Exists?
no
Return 404 Not Found
"Appointment not found"
Check current status allows check-in
Already checked_in/completed/cancelled?
yes
Return 400 Bad Request
"Invalid status transition"
Set status = checked_in
Record check_in_time
Save appointment
Return 200 OK
with updated appointment
Receive confirmation


Receptionist Check-In (Update Appointment Status)
Receptionist
API Gateway
Appointment Service
Login as receptionist
Select an appointment
Submit status update
(status = "checked_in")
Validate JWT (receptionist role)
Route to Appointment Service
Validate appointment exists
Exists?
no
Return 404 Not Found
"Appointment not found"
Check current status allows check-in
Already checked_in/completed/cancelled?
yes
Return 400 Bad Request
"Invalid status transition"
Set status = checked_in
Record check_in_time
Save appointment
Return 200 OK
with updated appointment
Receive confirmation

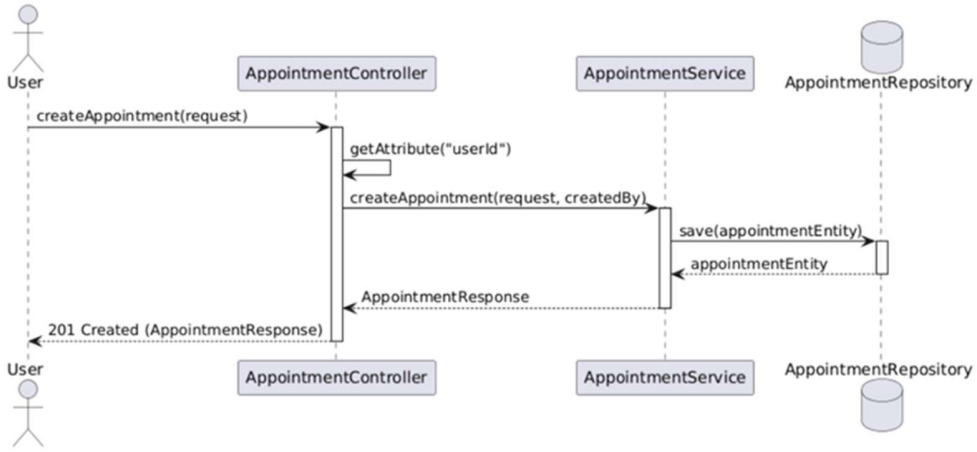

Centralized Error Handling
Client
API Gateway
Microservice
Send any API request
Public endpoint?
yes
no
Forward without token
Check Authorization header
Valid JWT?
no
Return 401 Unauthorized
Route to target service
Process request
Exception occurs?
yes
no
CustomException thrown
(BadRequest, NotFound, Forbidden, etc.)
Return success response
GlobalExceptionHandler catches it
Return appropriate HTTP status
(400, 404, 403, 409, etc.)
Receive JSON error or success data


Full Appointment Lifecycle
Patient/Receptionist
Appointment Service
Receptionist
Doctor
Medical Record Service
Patient
Book appointment
(see Booking Diagram)
Save with status "scheduled"
Patient arrives
Check-in appointment
(status = "checked_in")
Update status to "checked_in"
Set check_in_time
Consultation completed
Mark appointment as "completed"
Update status to "completed"
Create Medical Record
(via Medical Record Service)
Save diagnosis, prescription
Return 201
View own medical history
(via Medical Record Service)
Cancellation: At any point before completion,
an appointment can be cancelled (status → "cancelled" with reason).
The appointment remains in the database for audit purposes.

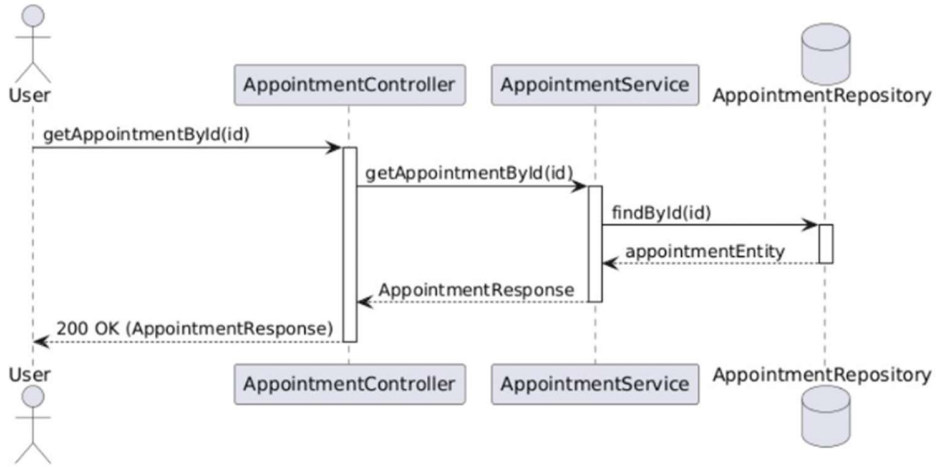
3- Sequence Diagrams

3.1 Appointment

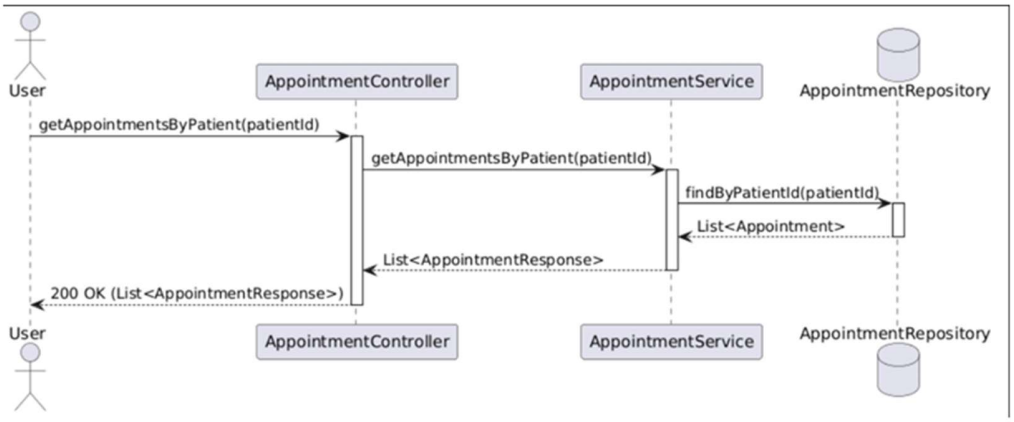
1) Create Appointment



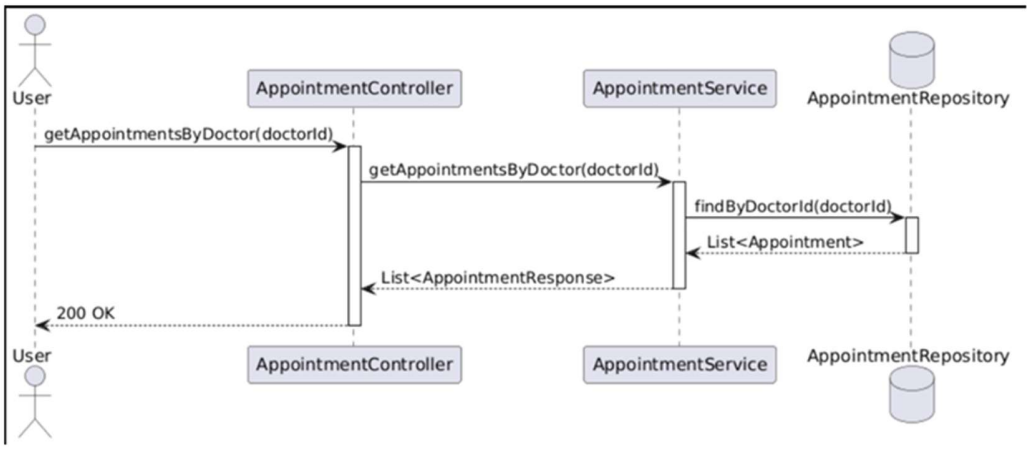
2) Get Appointment By Id



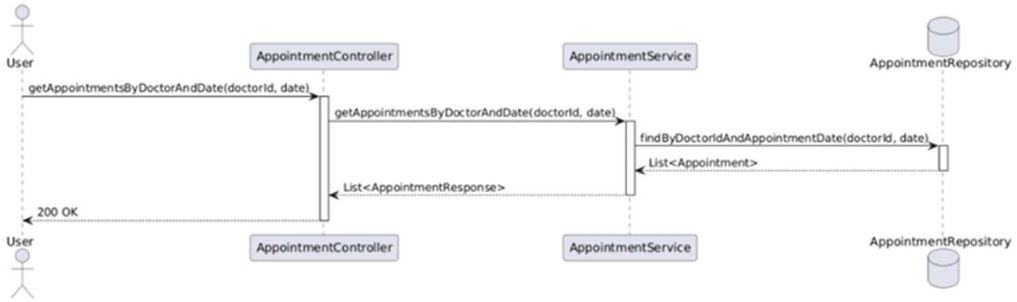
3) Get Appointments By Patient



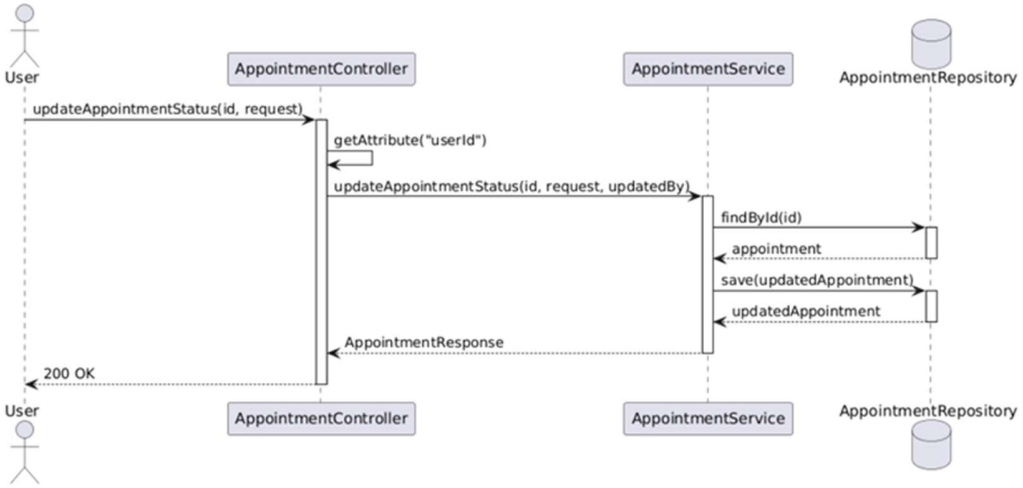
4) Get Appointments By Doctor



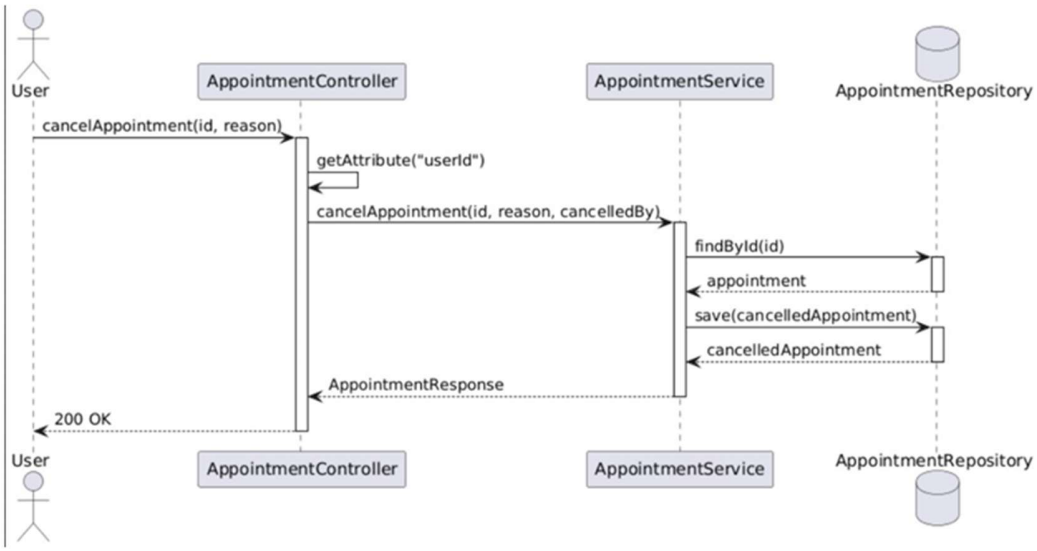
5) Get Appointments By Doctor And Date



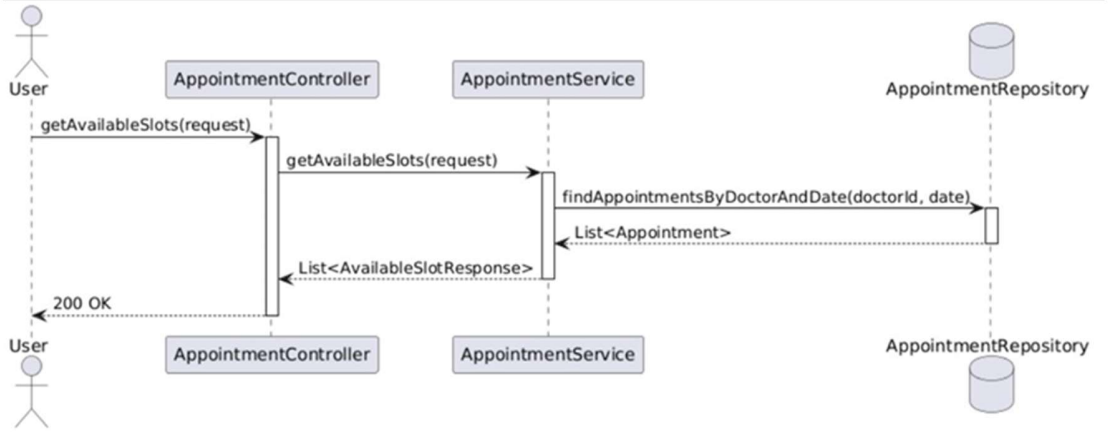
6) Update Appointment Status



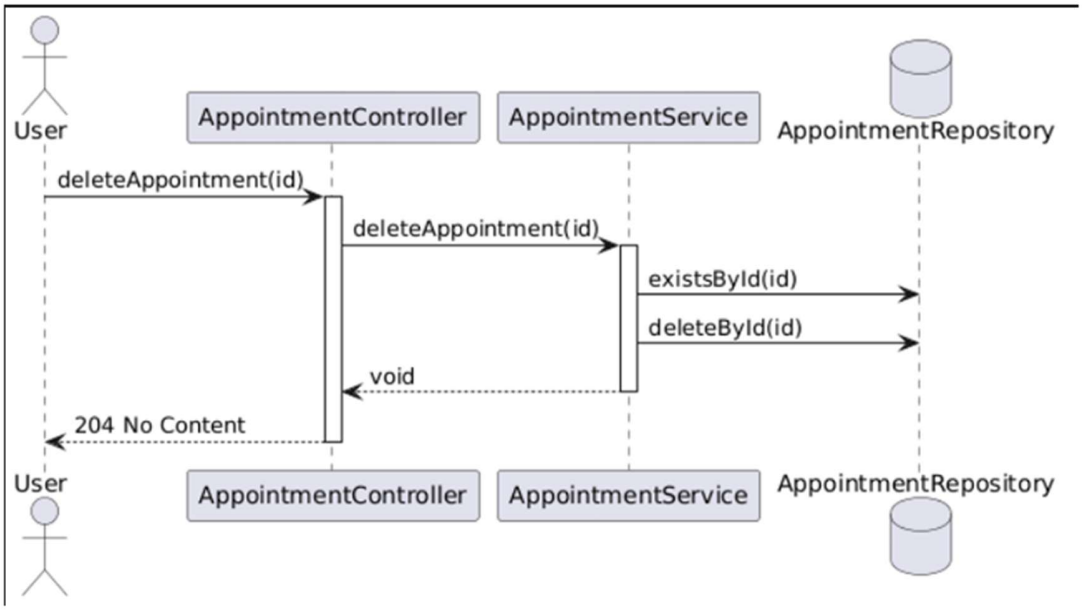
7) Cancel Appointment



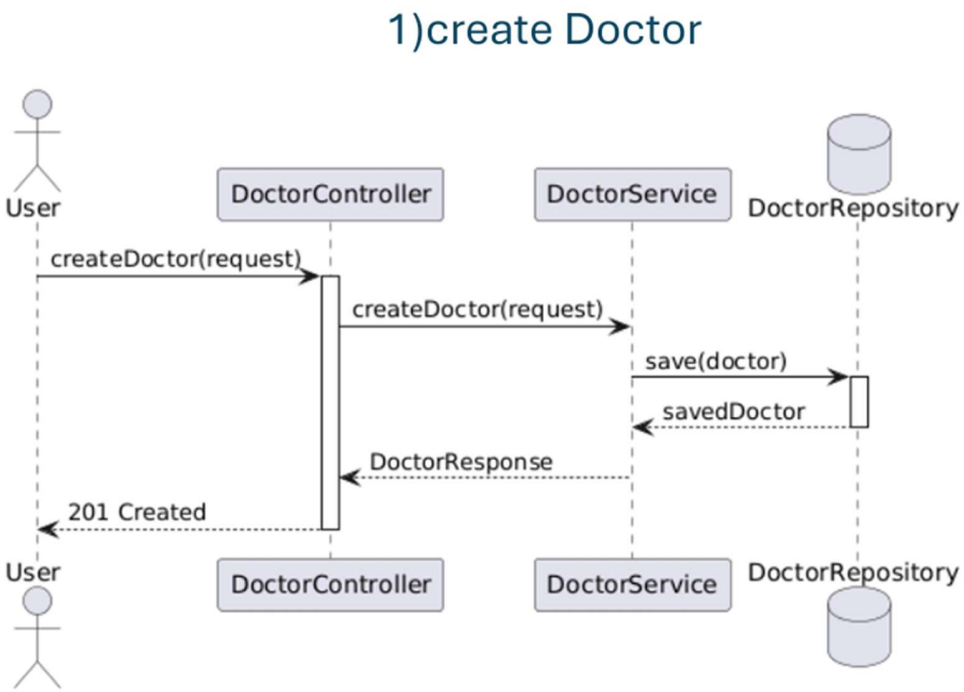
8) Get Available Slots

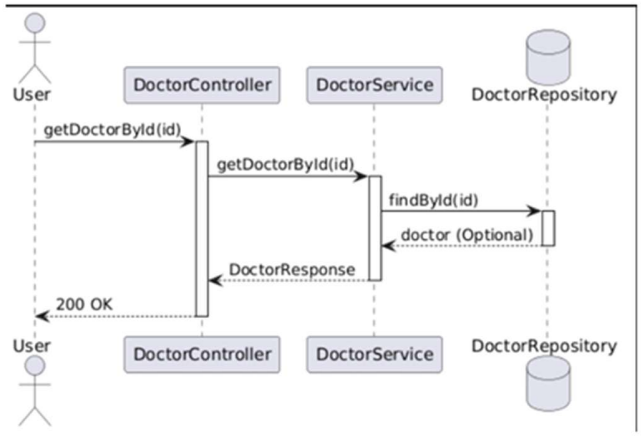


9) Delete Appointment

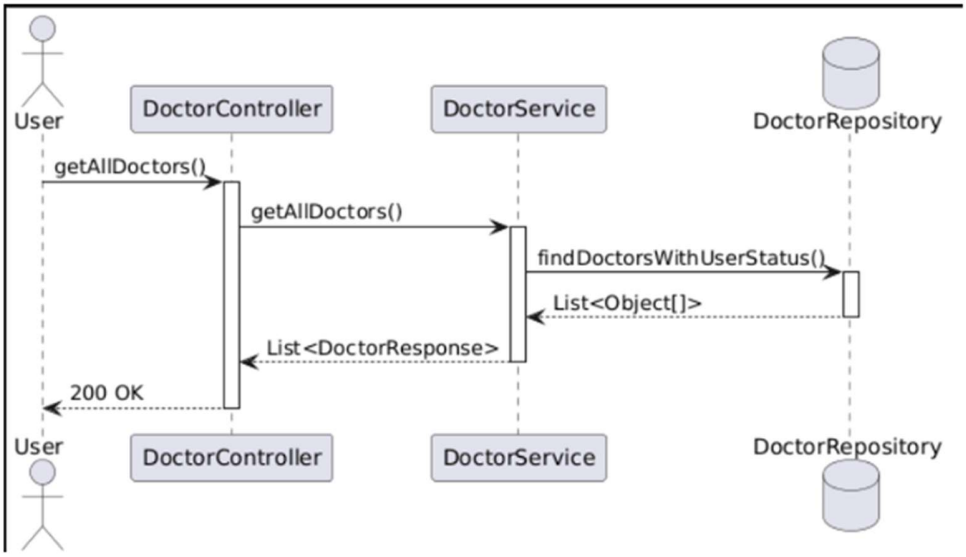


3.2 Doctor

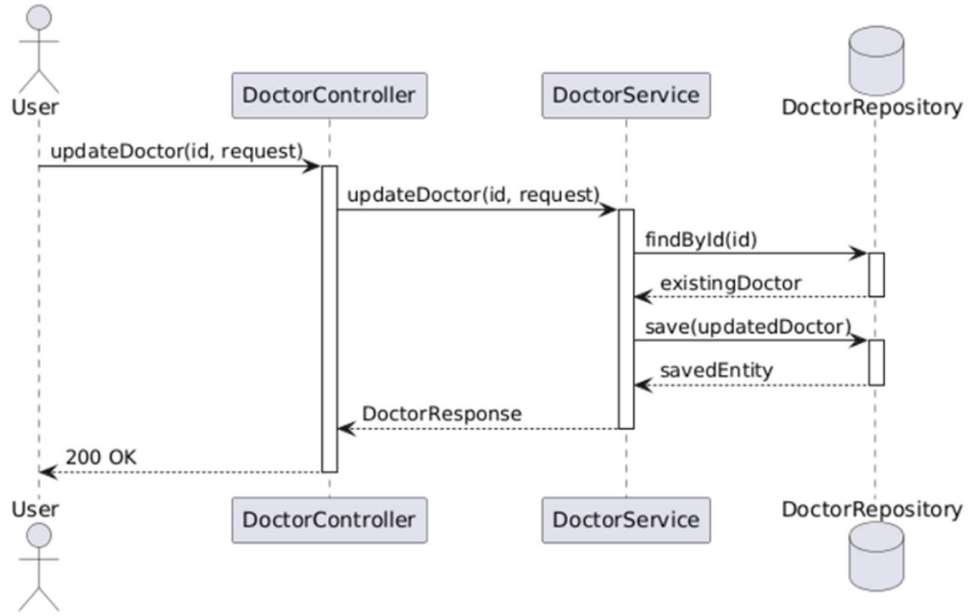

1)create Doctor
User
DoctorController
DoctorService
DoctorRepository
createDoctor(request)
createDoctor(request)
save(doctor)
savedDoctor
DoctorResponse
201 Created
User
DoctorController
DoctorService
DoctorRepository


2) Get Doctor By Id
User
DoctorController
DoctorService
DoctorRepository
getDoctorById(id)
getDoctorById(id)
findById(id)
doctor (Optional)
DoctorResponse
200 OK
User
DoctorController
DoctorService
DoctorRepository

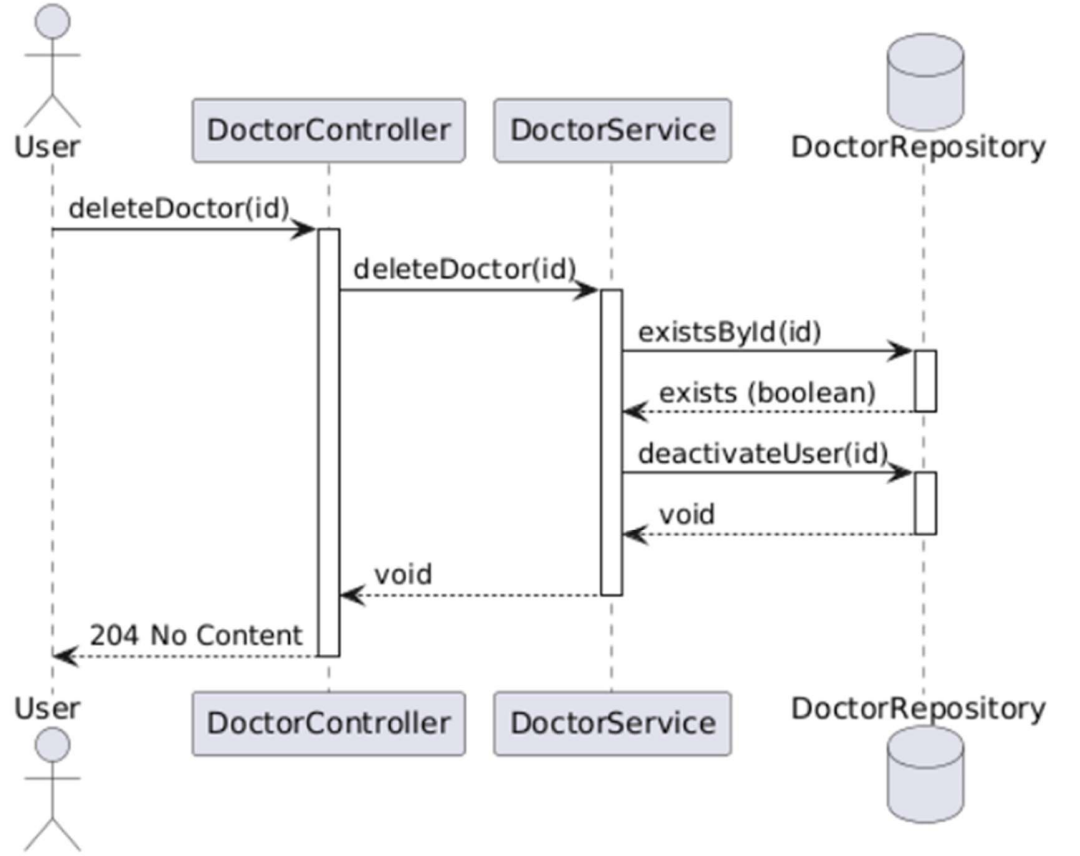
3) get All Doctors



4) update Doctor

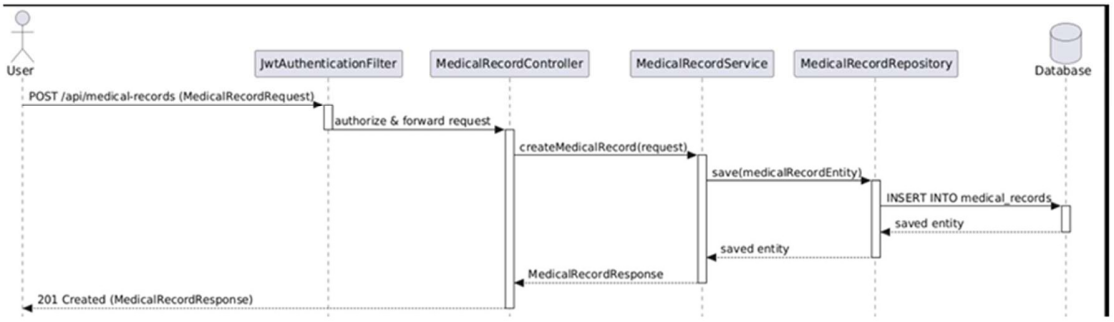


5) delete Doctor

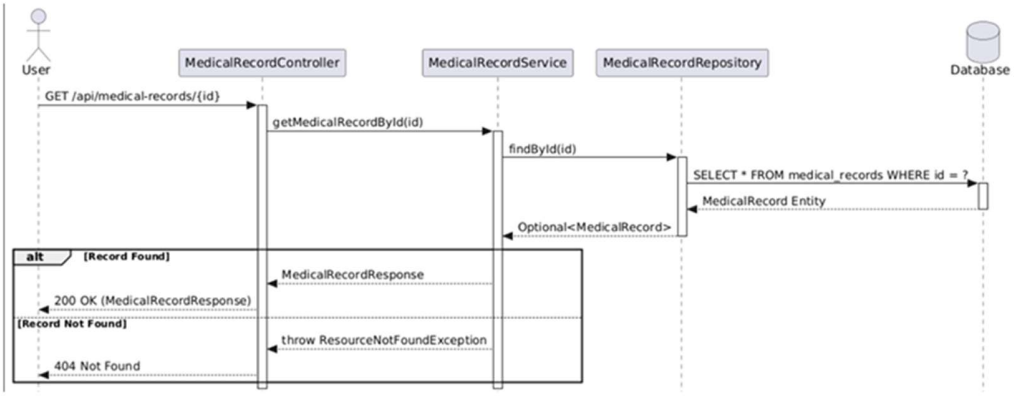


3.3 Medical Record

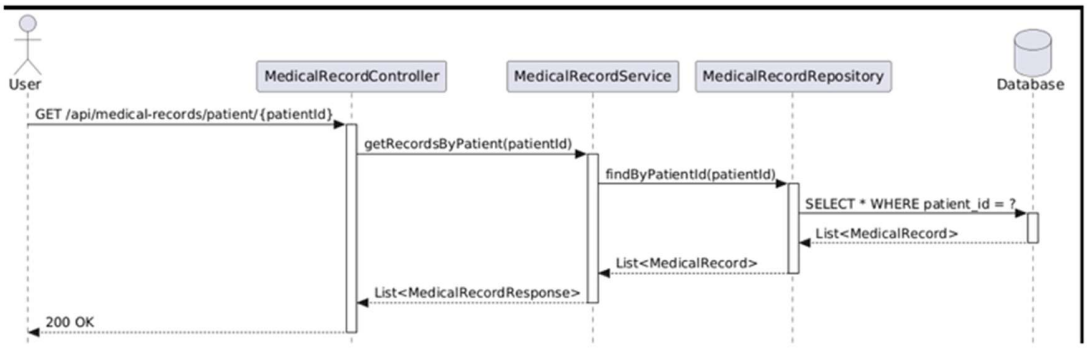
1)Create Medical Record



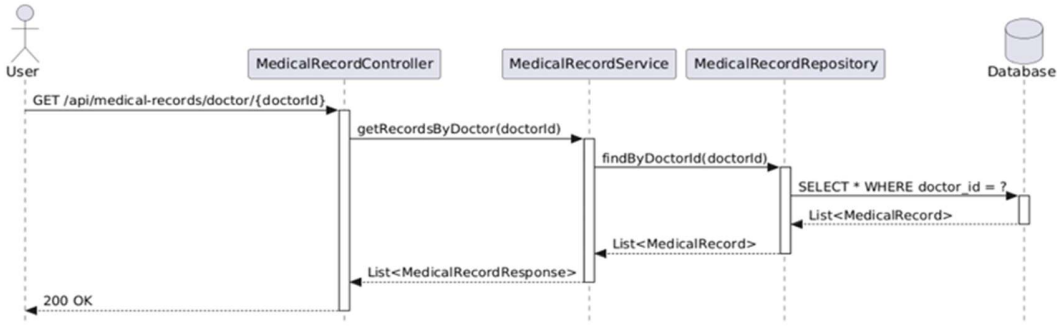
2) Get Medical Record By Id



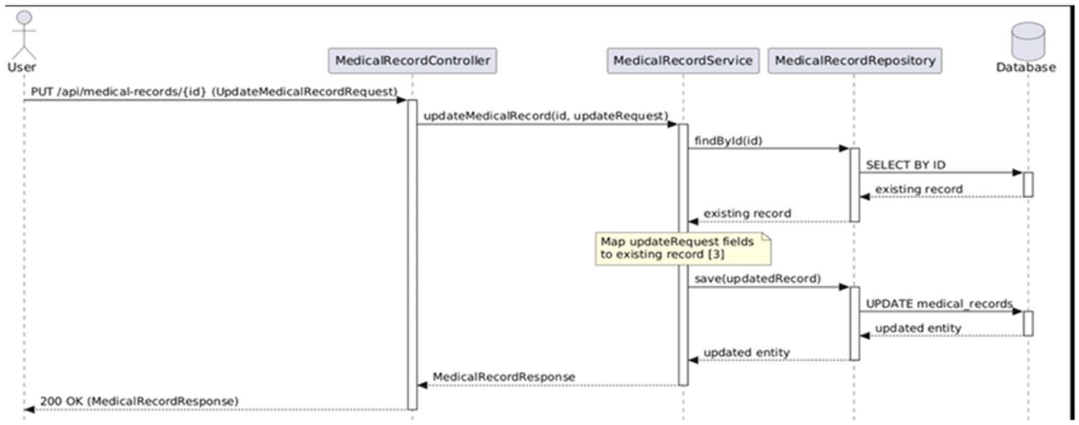
3) Get Medical Records By Patient



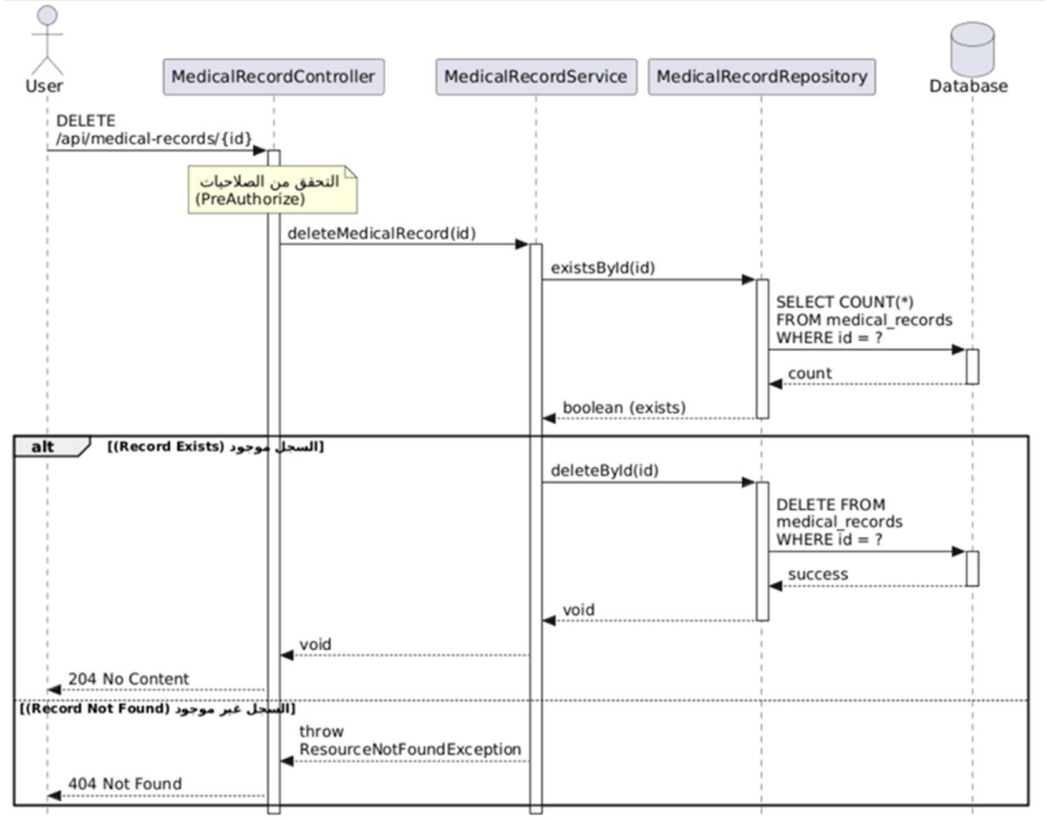
4) Get Medical Records By Doctor



5) Update Medical Record

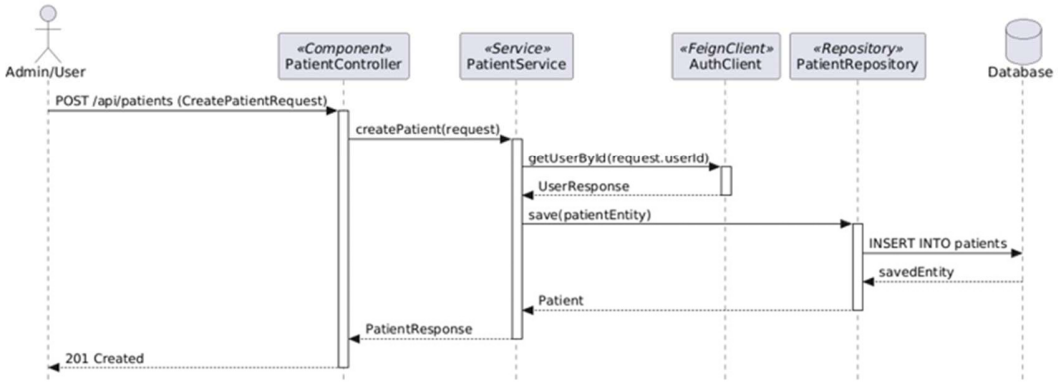


6) Delete Medical Record

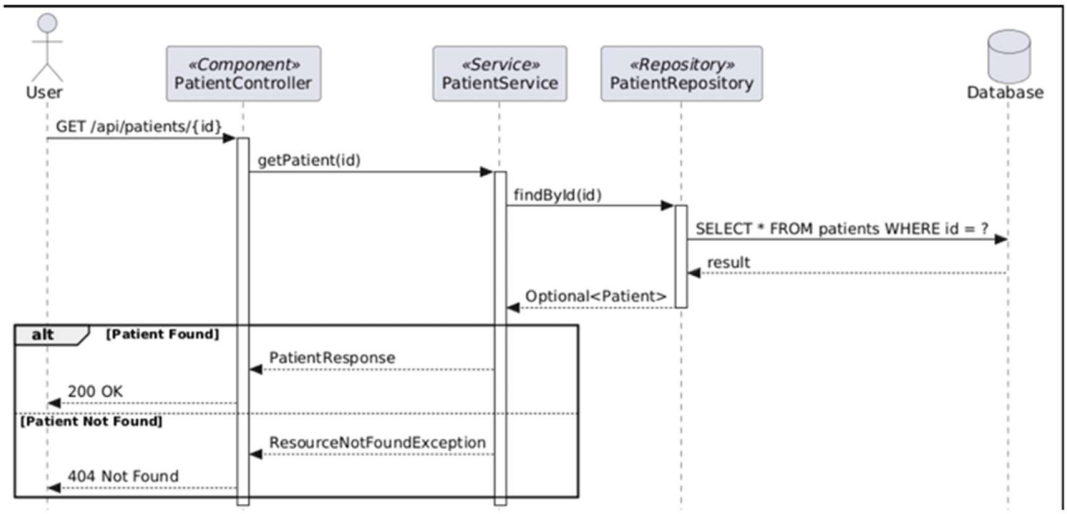


3.4 Patient

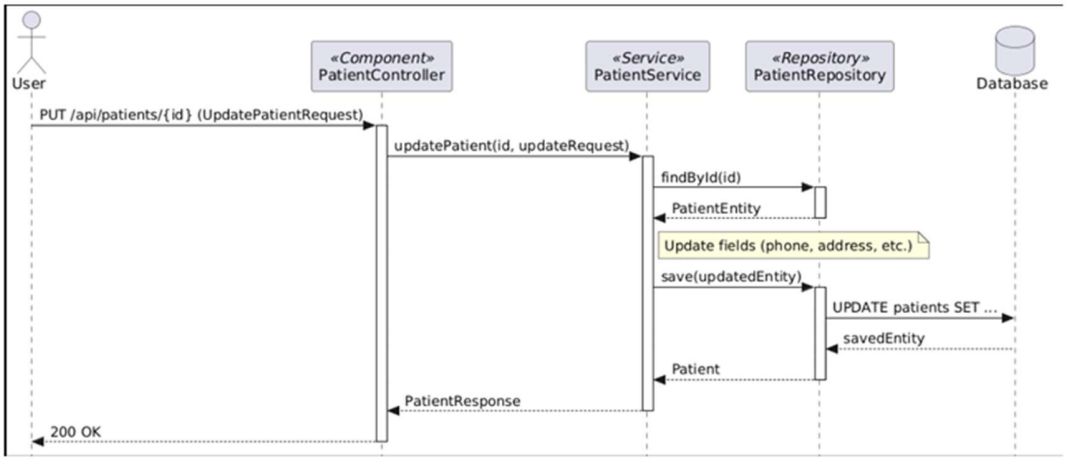
1) Create Patient



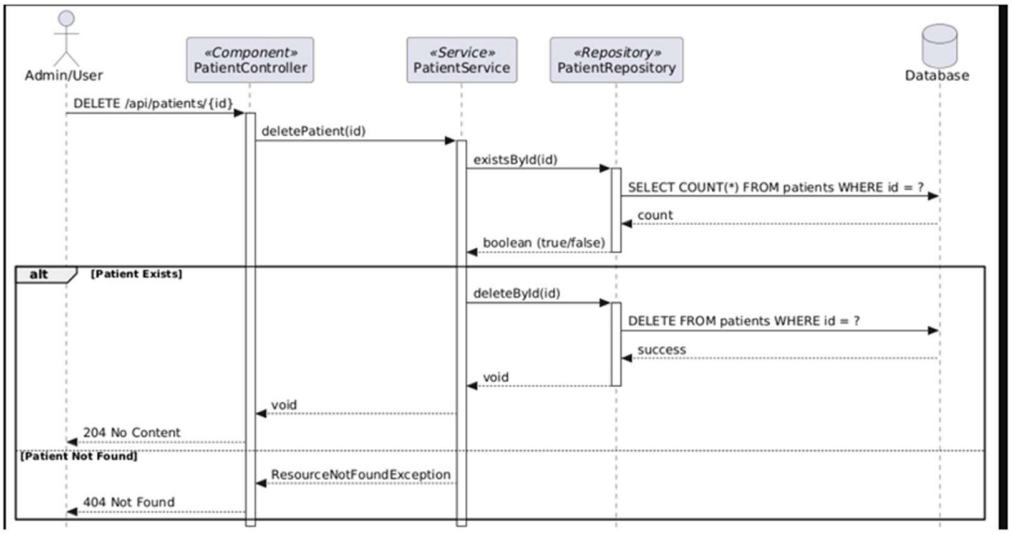
2) Get Patient By Id



3) Update Patient

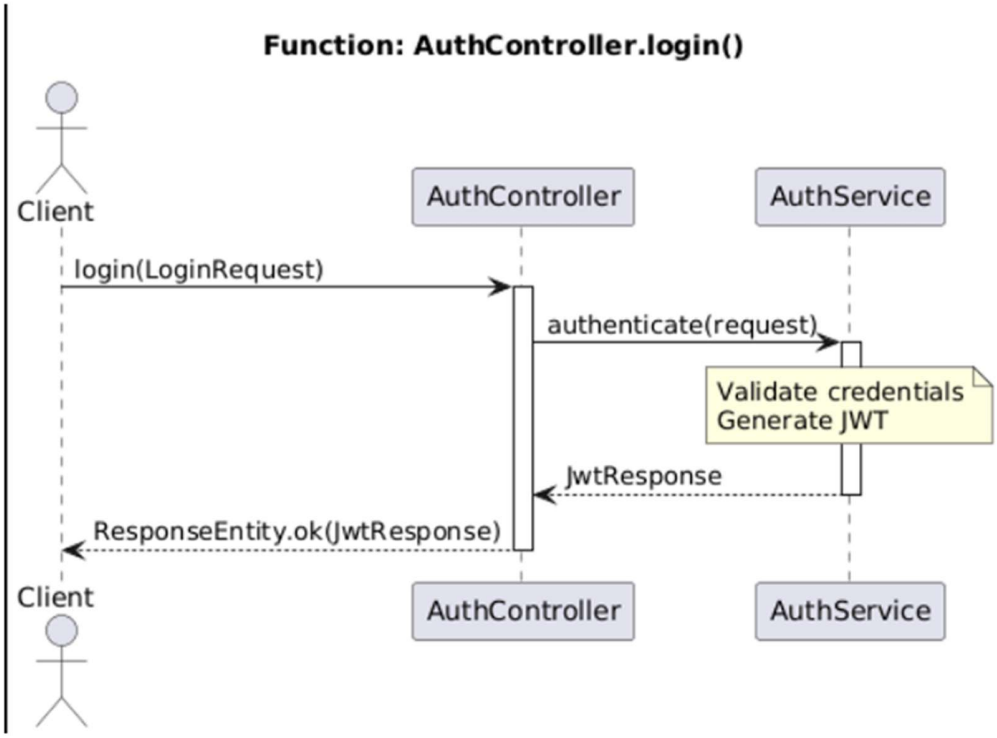


4) Delete Patient

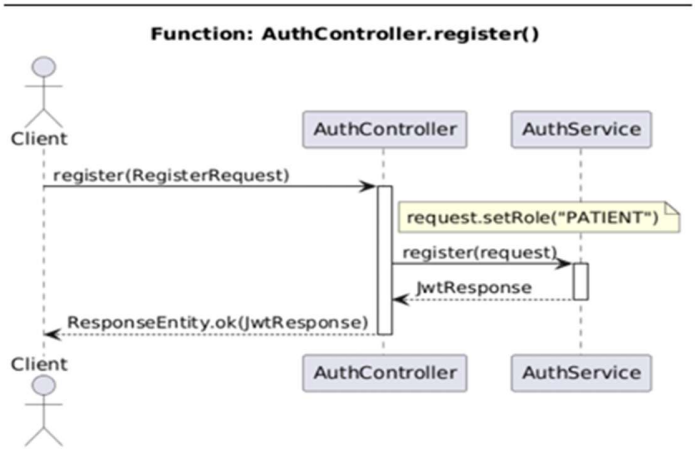


3.5 Auth

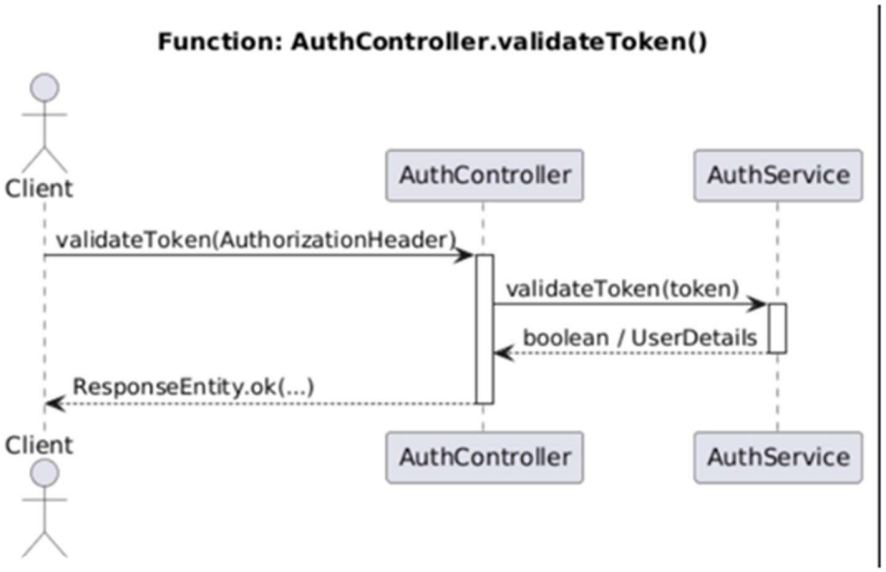
1)login



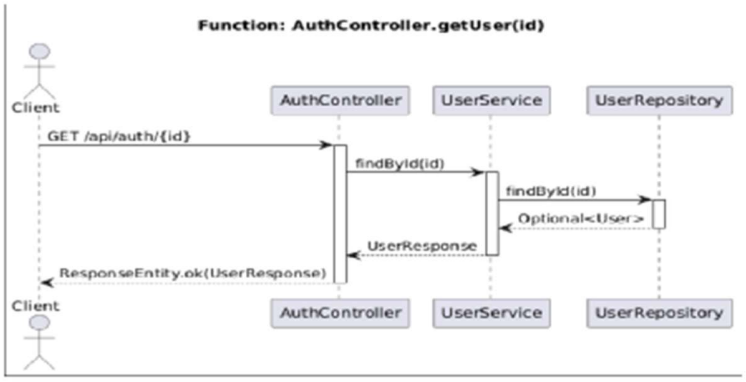
2) register



3) Validate Token

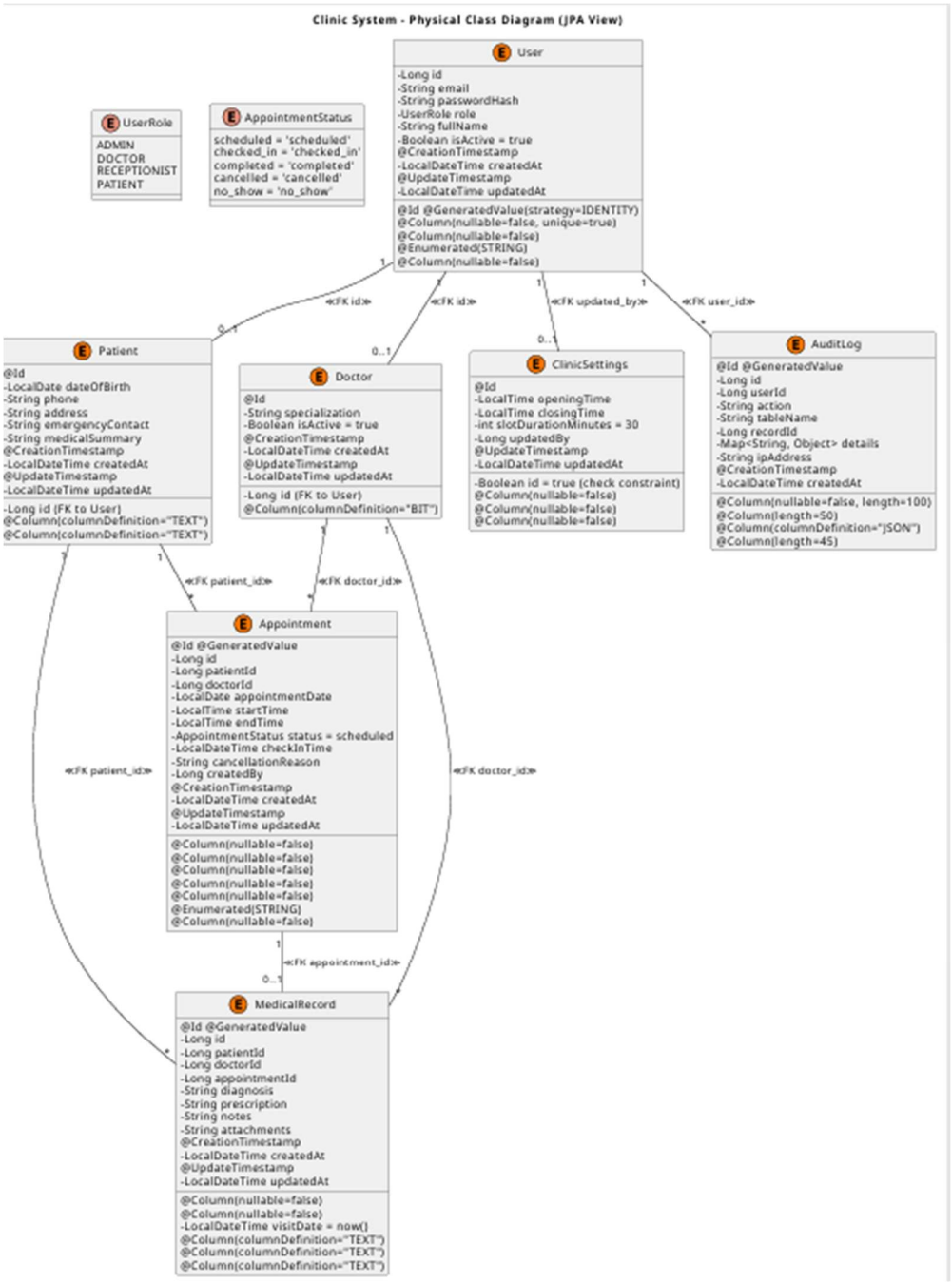


4) Get User By Id

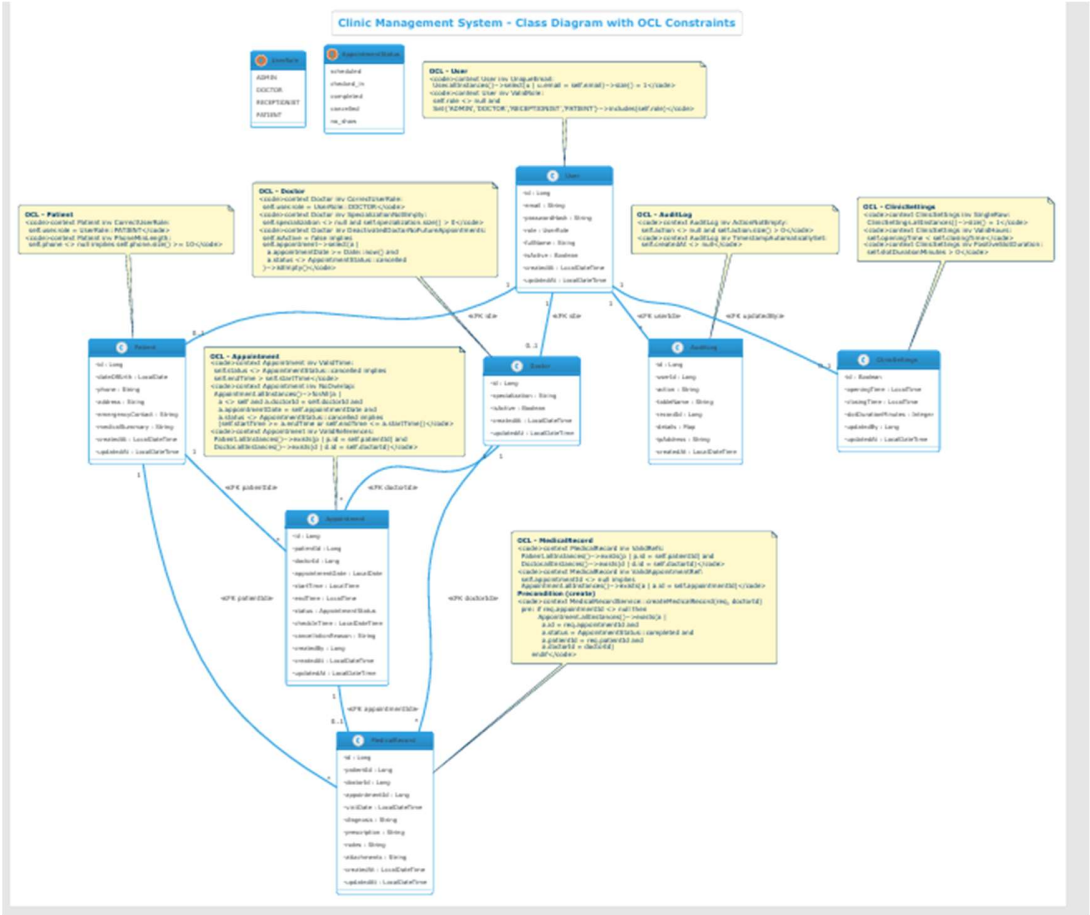


4- Class Diagram

4.1 Class



4.2- Class with OCL



5- ERD Diagram